ULUSLARARASI KIBRIS ÜNİVERSİTESİ
CYPRUS INTERNATIONAL UNIVERSITY

Project Title: Vertex KANBAN System

CMPE314 - Software Engineering

Submitted To: Behnam Shahriari

Group Number: 1

Submitted By:

Baran Koç - 22314825

Efe Kemal Kayış - 22314842

Berk Elmalı - 22314621

Emirhan Keskin - 22314856

Date of Submission:

21.05.2026

Table of Contents

Chapter 1: Introduction

Vertex KANBAN System is a browser-based task management application built using HTML5, CSS3, and Vanilla JavaScript. It provides teams with a real-time Kanban board to organise, track, and collaborate on tasks across three workflow stages: To Do, In Progress, and Done.

Github Link

1.1.Project Overview & Problem Statement

Modern software teams struggle with task visibility and coordination without a lightweight collaboration tool. Existing tools like Jira or Trello are heavyweight and require server infrastructure. Vertex KANBAN addresses this gap by providing a fully client-side Kanban board that runs entirely in the browser with no backend dependencies, making it instantly accessible without installation or account overhead for small teams and students.

1.2.Project Scope & Objectives

IN SCOPE:
- User registration and authentication (localStorage-based)
- Project creation, switching, and deletion
- Kanban board with three columns: To Do, In Progress, Done
- Task CRUD (create, read, update, delete) with priority, due date, and multi-assignee support
- Drag-and-drop task movement between columns
- Search and filter tasks by priority and assignee
- Notification system with in-app activity feed and toast messages
- Fully responsive design for desktop and mobile

OUT OF SCOPE:
- Server-side data persistence or cloud synchronisation
- Real-time multi-user collaboration
- Email/SMS notifications
- File attachments on tasks

1.3.Report Structure

This report is organised as follows: Chapter 2 covers project management including team roles and work breakdown. Chapter 3 identifies stakeholders and elicitation techniques. Chapter 4 specifies functional and non-functional requirements. Chapter 5 presents system models (use cases, activity diagrams, sequence diagrams, class diagram, and DFDs). Chapter 6 describes the implementation. Chapter 7 covers testing. Chapter 8 concludes with lessons learned.

Chapter 2: Project Management

2.1 Team Members & Roles

Name	Student ID	Role	Responsibilities
Baran Koç	22314825	Project Manager	Requirements gathering, use case analysis, final deployment, documentation
Efe Kemal Kayış	22314842	Lead Developer / Backend	System architecture, database design, backend logic, API integration
Berk Elmalı	22314621	Frontend Developer	UI/UX prototyping, frontend development, bug fixing
Emirhan Keskin	22314856	Support & Testing	Dev environment setup, unit/acceptance testing, deployment support,documentation

2.2 Work Breakdown & Individual Contributions

Task ID	Task Name / Description	Primary Member(s)	Contributor(s)
1	Requirements Gathering: Define system goals and collect stakeholder needs	Baran	All members
2	Use Case Analysis: Create UML use case diagrams	Baran	Efe
3	System Architecture Design: Design overall system architecture and tech stack	Efe	Baran
4	Database Design: Design MongoDB/localStorage schema for tasks, projects, users	Berk, Efe	
5	UI/UX Prototyping: Create wireframes and mockups for the Kanban interface	Berk	Emirhan
6	Setup Dev Environment: Configure Git, Node.js, project structure for the team	Emirhan	
7	Frontend Development: Implement Kanban board with drag-and-drop functionality	Berk	Efe
8	Backend Development: Build Node.js REST API for task CRUD operations	Efe	Berk
9	API Integration: Connect frontend components with backend REST API endpoints	Efe, Berk	
10	Unit Testing: Write unit tests for core task management modules	Berk, Emirhan	
11	User Acceptance Testing: Test with end users and gather feedback	Emirhan, Baran	
12	Bug Fixing: Fix bugs found during integration and user testing	Efe, Berk	
13	Deployment Preparation: Prepare deployment scripts and environment configuration	Emirhan, Baran	
14	Final Deployment: Deploy the application to the production environment	Baran	
15	Project Documentation: Write final project report, user manual, and deployment guide	Baran, Emirhan	

2.3 Chosen Software Process Model & Justification

We adopted an Agile / Iterative development process model. Given the rapidly evolving requirements of a task management tool and the small team size (4 members), Agile allowed us to deliver working increments early (e.g., authentication first, then board, then notifications), adapt to feedback between iterations, and distribute tasks flexibly. A strict Waterfall model would have been too rigid for a UI-heavy project where design decisions emerge during implementation.

2.4 Project Timeline

The project followed an Agile schedule spanning from 8 March 2026 to 19 April 2026 (six weeks). The Gantt chart below shows all 16 tasks with their durations and dependencies. A resource-based view is also provided, grouping tasks by assigned team member.

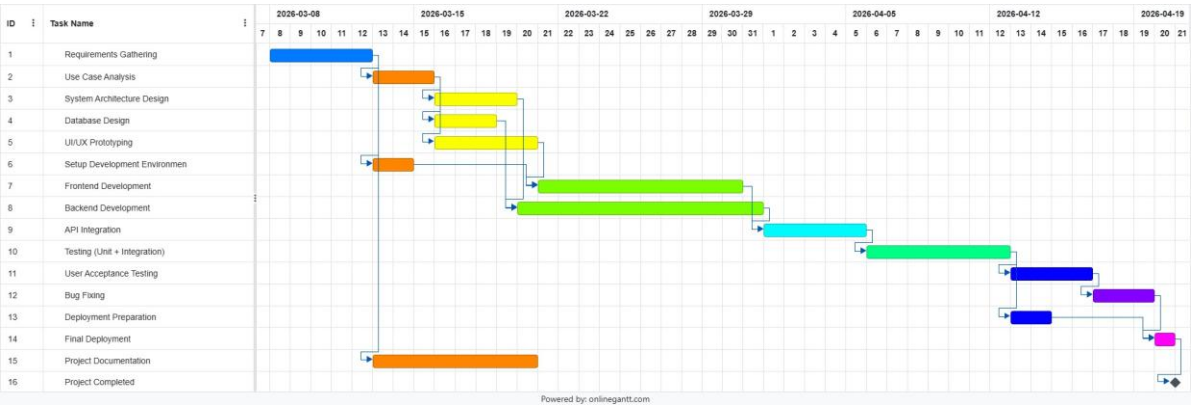


Figure 2.1 — Project Gantt Chart (Overall Timeline)

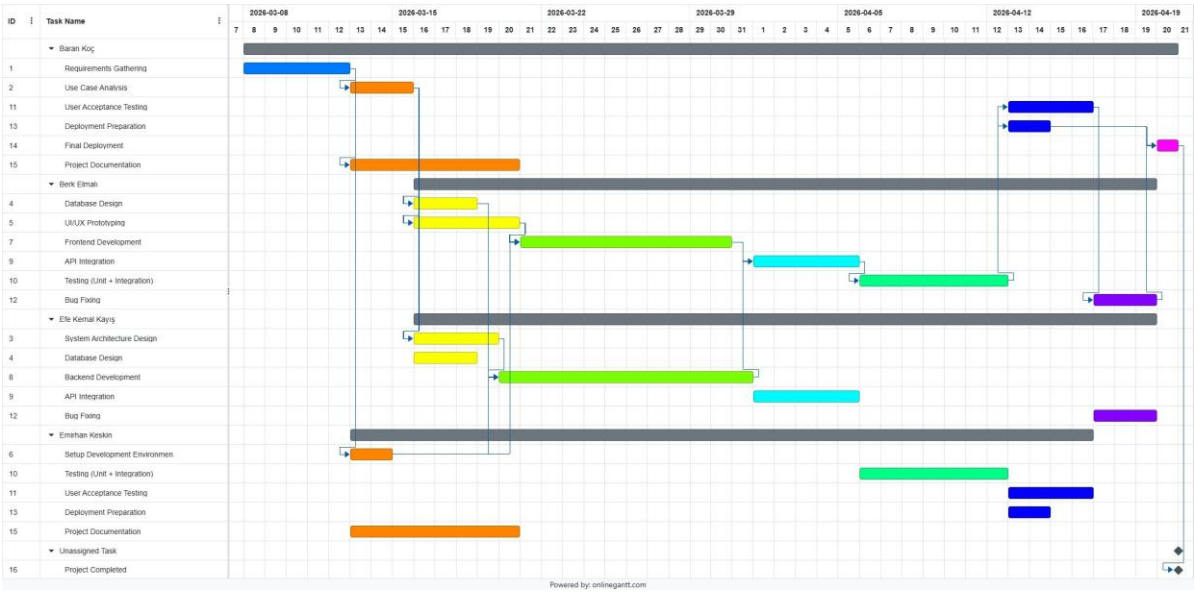


Figure 2.2 — Resource-Based Gantt Chart (Per Team Member)

Chapter 3: Stakeholder & Requirements Elicitation

3.1 Stakeholder Identification

The following stakeholders were identified for the Vertex KANBAN System:

- *Primary Stakeholders: Registered Users (students, small software teams) who directly create projects, manage tasks, and interact with the Kanban board daily.*
- *Secondary Stakeholders: Course Instructor (CMPE314 — evaluates deliverables), University Administration (academic oversight), and future developers who may extend the system.*

3.2 Stakeholder Analysis

Stakeholder	Interest	Influence	Key Requirements
Registered User	Track and manage tasks efficiently	High	Intuitive UI, fast task creation, drag-and-drop
Project Owner	Oversee team productivity	High	Project CRUD, member management, notifications
Course Instructor	Assess software engineering practices	Medium	Full requirements coverage, proper documentation
University Admin	Academic compliance	Low	GDPR / data privacy considerations

3.3 Requirements Elicitation Techniques Used

The following elicitation techniques were used:
1. Brainstorming Sessions: The four team members held joint planning meetings to define core features and workflow based on shared experience with task management tools.
2. User Story Mapping: User stories were written from the perspective of a typical team member (e.g., 'As a user, I want to drag a task card to Done so that my progress is immediately visible.').
3. Competitive Analysis: Existing tools (Trello, GitHub Projects) were surveyed to identify must-have features and gaps.
4. Prototype Walkthroughs: An interactive HTML prototype was built early and reviewed iteratively by all team members.

Chapter 4: Requirements Specification

4.1 Functional Requirements (FRs)

- *Format Example:*
  - **ID:** FR-001
  - **Title:** User Registration
  - **Description:** The system shall allow a new user to register for an account by providing a valid email address, full name, and a password.

4.2 Non-Functional Requirements (NFRs)

- *Performance:* "The system shall load the dashboard in under 2 seconds."
- *Security:* "All user passwords shall be stored in an encrypted format."
- *Usability:* "The interface shall be intuitive enough for a non-technical user to learn basic functions within 10 minutes."

Chapter 5: System Modelling

The following subsections present the system models produced for Vertex KANBAN System. All diagrams were produced as part of the CMPE314 lab deliverables.

5.1 Use Case Model

5.1.1 Use Case Diagram

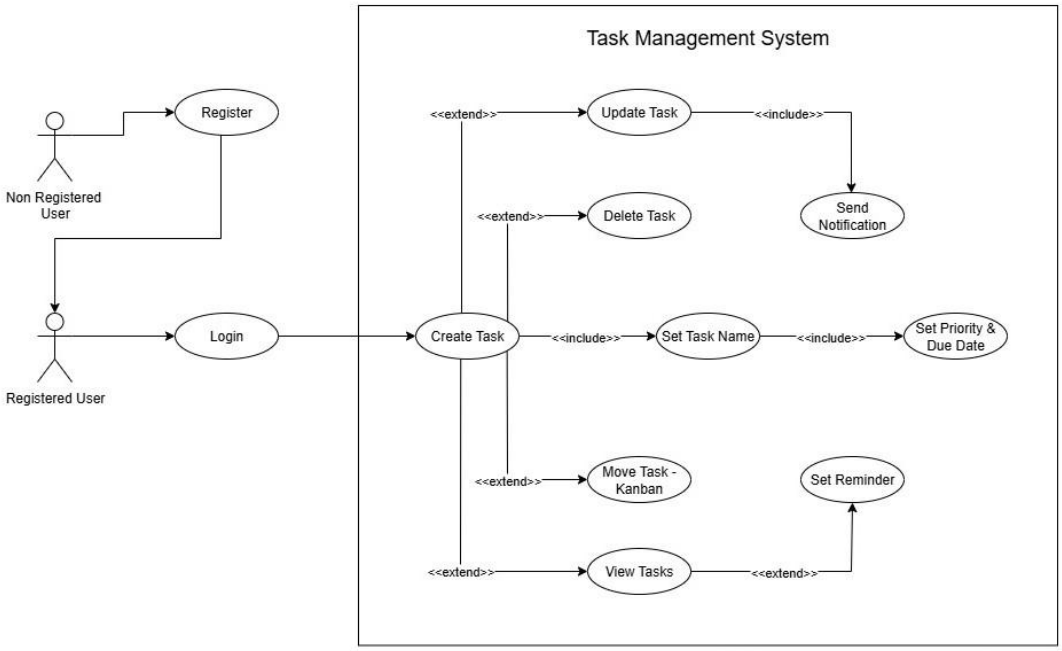


Figure 5.1 — Use Case Diagram: Vertex KANBAN System

5.1.2 Detailed Use Case Descriptions:

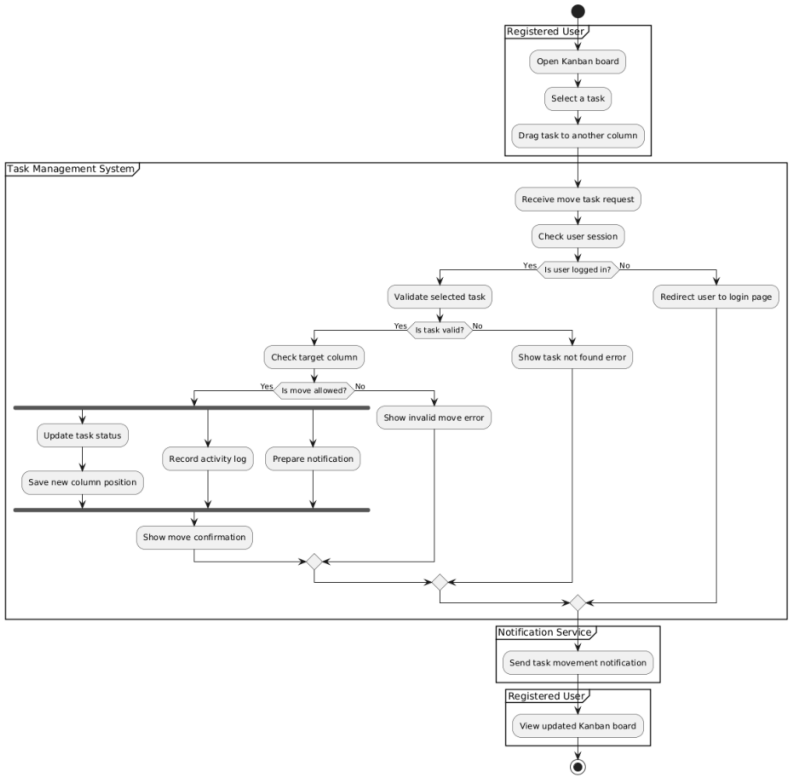
The use case diagram identifies three primary actors: Guest (unauthenticated visitor), Registered User, and Project Owner. Key use cases include: Register, Login, Logout, Create Project, Delete Project, Create Task, Edit Task, Delete Task, Move Task, Search Tasks, Filter Tasks, and View Notifications.

- *Use Case Name:* Login
- *Actor:* User
- *Pre-condition:* User has a registered account.
- *Post-condition:* User is authenticated and granted access.
- *Main Success Scenario:* 1. User enters email and password. 2. System validates credentials. 3. System redirects to dashboard.
- *Alternate Flows:* 2a. Invalid credentials: System displays an error message.

5.2 Dynamic Models

5.2.1 Activity Diagrams

An activity diagram was produced for the "Move Task on Kanban Board" workflow. The diagram captures the full lifecycle: the registered user opens the board, selects a task, and drags it to another column. The Task Management System then checks the user session (Decision: logged in / redirect to login) and validates the selected task (Decision: task valid / show error). If the move is allowed (Decision), the system forks into three parallel branches — Update Task Status, Record Activity Log, and Prepare Notification — before joining and showing a move confirmation. The Notification Service then sends the task movement notification, and the user views the updated board.



Description:
Parallelism: Fork section after the move is allowed, where the system updates the task status, saves the new column position, records the activity log, and prepares the notification.
Decision : Yes/No branches, similar to if-else statements. The system follows only one path depending on the condition.

Figure 5.2 — Activity Diagram: Move Task on Kanban Board

Design Note: *The Notification Service is depicted as an external participant in the diagram. In the actual implementation, notifications are handled internally within the browser using localStorage — there is no separate external notification service. This diagram was produced during the design phase before the final tech stack was confirmed.*

5.2.2 Sequence Diagrams

A sequence diagram was produced for the "Create Task" workflow. Actors and participants: Registered User, Task Management System, Authentication Service, Task Service, Board Service, Task Records, Kanban Board Data, and Notification Service. The diagram models three alternative scenarios (alt fragment): (1) Valid task details — the system validates the session, the user submits task details, the Task Service saves the record, the Board Service places the task in the "To Do" column, and optionally the Notification Service creates a reminder if a due date is set; (2) Missing or invalid task details — a validation error is shown; (3) User session expired — the system redirects to the login page.

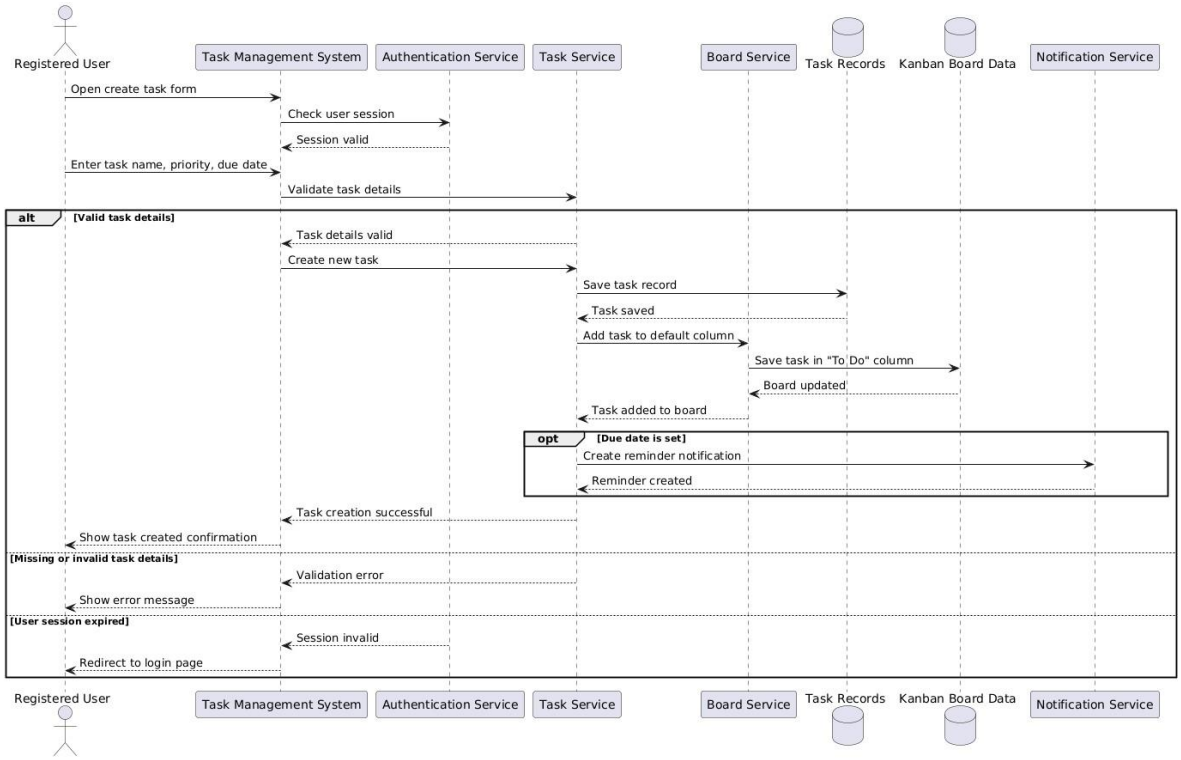


Figure 5.3 — Sequence Diagram: Create Task

Design Note: *The diagram references backend microservices (Authentication Service, Task Service, Board Service) and separate databases (Task Records, Kanban Board Data) that do not exist in the final implementation. The actual system is a single-page application using browser localStorage for all persistence — no server-side backend was built. The diagram reflects early architectural assumptions that were revised once the technology stack was finalised as pure client-side HTML/CSS/JavaScript.*

5.3 Structural Model

5.3.1 Class Diagram

The system is structured around five IIFE (Immediately Invoked Function Expression) modules, each representing a logical class:

AuthManager: getUsers(), saveUsers(), register(name, email, password), login(email, password), logout(), getCurrentUser(), getInitials(name)

ProjectManager: getAll(), getByUser(userId), create(name, desc, color, ownerId), deleteProject(id), getActive(), setActive(id), addMember(projectId, memberName)

TaskManager: getAll(), getByProject(projectId), create(data), update(id, data), moveTask(id, newStatus), deleteTask(id), deleteByProject(projectId), isOverdue(task)

BoardRenderer: render(), renderCard(task), setSearch(q), setPriorityFilter(v), setAssigneeFilter(v), initDragAndDrop()

NotificationManager: add(notif), markAllRead(), clearAll(), renderPanel(), showToast(type, text)

5.4 Data Flow Model

5.4.1 Context Diagram

The Level-0 Context Diagram shows the Kanban Task Management System as a single process interacting with external entities. The Registered User submits Login Details, a View Board Request, a Search Keyword, and Filter Options. The system returns the Login Status, Task List, Filtered Results, Kanban Board View, Error Messages, and Reminder/Notification Messages. The system authenticates via the User Accounts database, retrieves data from Task Records and Kanban Board Data, sends Notification/Reminder Requests to the Notification Service (which replies with a Notification Status), and stores Notification History in Notification Records.

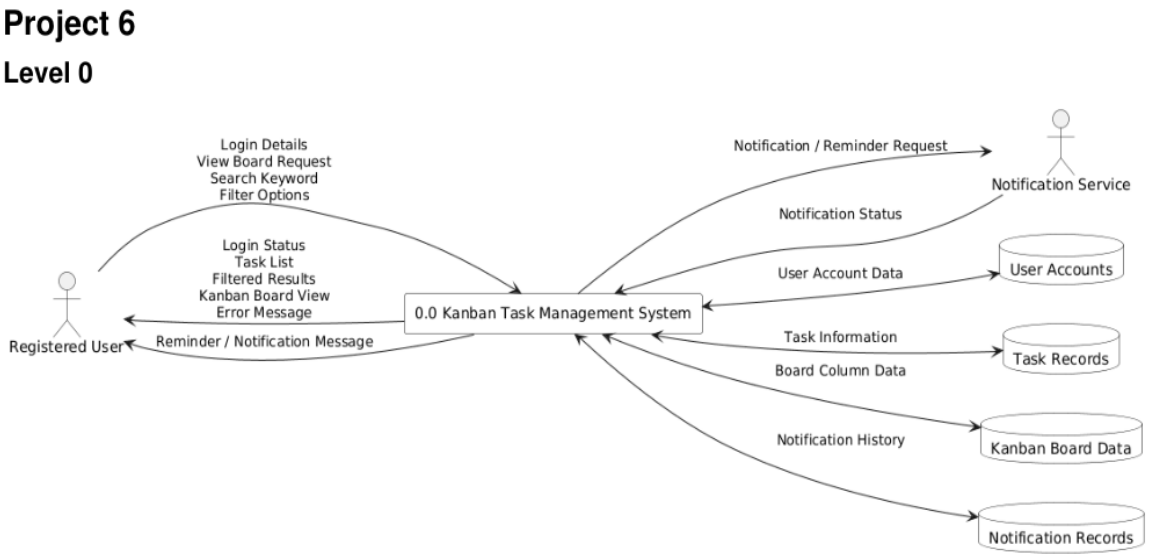
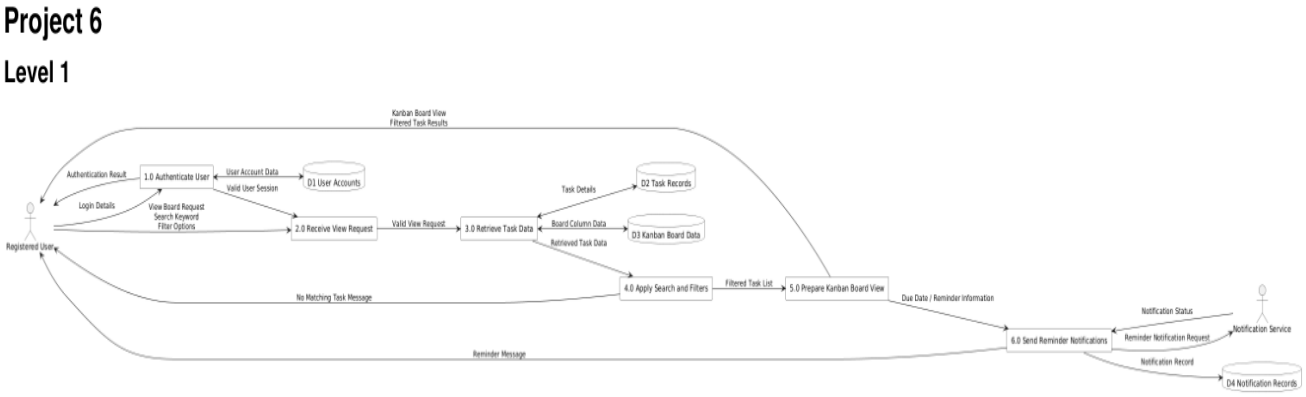


Figure 5.4 — Level-0 Context Diagram (DFD)

Design Note: *The Context Diagram shows Task Records and Kanban Board Data as separate external data stores, and the Notification Service as an external entity. In the actual system, all data (users, projects, tasks, board state, notifications) is stored in a single browser localStorage environment with no external services. This mismatch reflects design decisions made before the final single-page, client-only architecture was adopted.*

5.4.2 Level-1 DFD

The Level-1 DFD decomposes the system into six processes: (1.0) Authenticate User — validates login details against User Accounts and issues a valid session; (2.0) Receive View Request — accepts the board view request with search keywords and filter options; (3.0) Retrieve Task Data — queries Task Records and Kanban Board Data for task details and board column data; (4.0) Apply Search and Filters — filters retrieved data and returns a filtered task list; (5.0) Prepare Kanban Board View — assembles the filtered list into the board view and checks for due dates/reminders; (6.0) Send Reminder Notifications — forwards reminder requests to the Notification Service and records history in Notification Records.

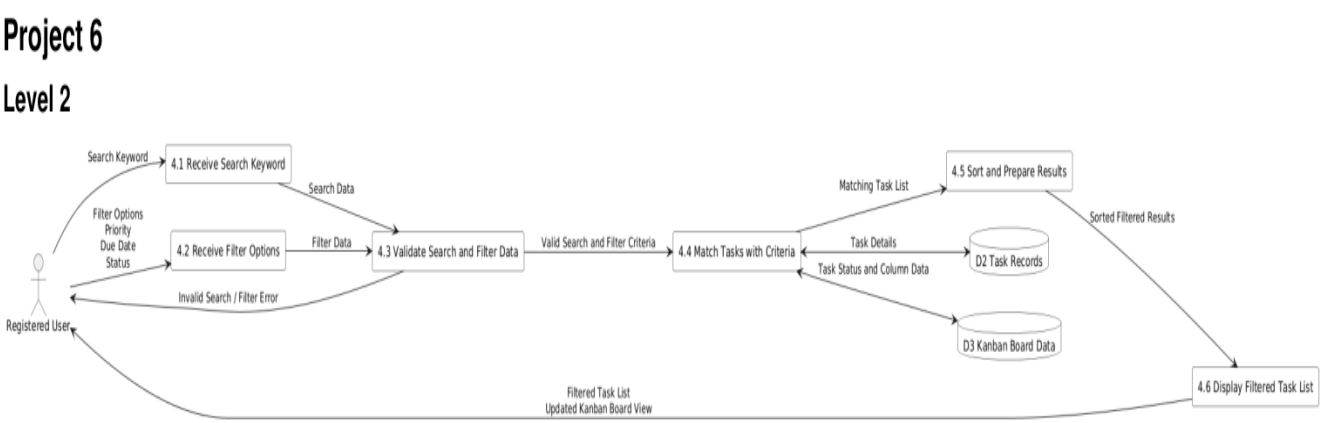


Description: The system is decomposed into six processes. Process 1.0 (Authenticate User) validates the user's login details against the User Accounts database and issues a valid session. Process 2.0 (Receive View Request) accepts the board view request together with search keywords and filter options, then forwards a validated view request. Process 3.0 (Retrieve Task Data) queries Task Records and Kanban Board Data to obtain task details and board column data. Process 4.0 (Apply Search and Filters) filters the retrieved data according to the user's criteria and returns a filtered task list. Process 5.0 (Prepare Kanban Board View) assembles the filtered list into a Kanban board view and checks for due date or reminder information. Process 6.0 (Send Reminder Notifications) forwards reminder requests to the Notification Service and records notification history in Notification Records.

Figure 5.5 — Level-1 DFD

Design Note: *The Level-1 DFD inherits the same architectural mismatch as the Context Diagram: the six processes assume distinct backend services and separate data stores, whereas the actual implementation consolidates all logic into JavaScript IIFE modules (AuthManager, ProjectManager, TaskManager, NotificationManager) operating on a unified localStorage store.*

Level-2 DFD — Process 4.0 (Apply Search and Filters) is further decomposed into six sub-processes: (4.1) Receive Search Keyword; (4.2) Receive Filter Options (priority, due date, status); (4.3) Validate Search and Filter Data — invalid entries return an error to the user; (4.4) Match Tasks with Criteria — queries Task Records and Kanban Board Data; (4.5) Sort and Prepare Results; (4.6) Display Filtered Task List and updated Kanban Board View.



Description: Process 4.0 (Apply Search and Filters) is broken down into six sub-processes. Sub-process 4.1 receives the search keyword entered by the user. Sub-process 4.2 receives filter options such as priority, due date, and status. Sub-process 4.3 validates the combined search and filter input; invalid entries produce an error message returned directly to the user. Sub-process 4.4 matches tasks against the validated criteria by querying Task Records and Kanban Board Data. Sub-process 4.5 sorts the matching results and prepares them for display. Sub-process 4.6 presents the filtered task list and the updated Kanban board view to the registered user.

Figure 5.6 — Level-2 DFD: Process 4.0 Apply Search and Filters

Design Note: *The Level-2 DFD lists filter options of priority, due date, and status (sub-process 4.2), but the live interface also supports filtering by Assignee/Member — a field added during development. Additionally, the diagram includes a dedicated sort-and-prepare step (sub-process 4.5), which was not implemented as a separate step; results are displayed in insertion order. Both differences emerged as requirements evolved after this diagram was created.*

Chapter 6: Implementation

6.1 Technology stack details

Vertex KANBAN System is built entirely with client-side web technologies. No backend server, framework, or third-party library was used. All data is persisted in the browser's localStorage. The codebase (JavaScript 51.6 %, CSS 27.5 %, HTML 20.9 %) is split into one HTML entry point, one stylesheet, and seven JavaScript IIFE modules.

Layer	Technology	Details
Markup	HTML5	Single-page entry point (index.html). Semantic structure for auth view, app view, modals, and three Kanban board columns.
Styling	CSS3 + Custom Properties	css/style.css. Dark-theme design system using CSS variables (--accent-cyan, --bg-card, etc.); responsive flexbox layout.
Logic	Vanilla JavaScript ES6+	7 IIFE modules (auth, project, task, notification, board, modal, app). No framework or build step required.
Persistence	Browser localStorage	6 named keys: vertex_users, vertex_session, vertex_projects, vertex_tasks, vertex_notifications, vertex_active_project.
Session flags	sessionStorage	Prevents duplicate overdue / reminder toast alerts within the same browser session per task.
ID generation	crypto.randomUUID()	RFC-4122 UUIDs for users, projects, and tasks. Falls back to Date.now() + Math.random() on older browsers.
Font	Inter via Google Fonts	UI typeface loaded with a CSS @import rule. No JavaScript dependency.

JavaScript Module Overview

auth.js — AuthManager

Handles user registration, login, and logout. Passwords are stored as a djb2-variant hash (base-36) in vertex_users; the active session token is held in vertex_session. Key methods: register(), login(), logout(), getCurrentUser(), getInitials().

project.js — ProjectManager

Full CRUD for projects persisted in vertex_projects. Tracks the currently open project via vertex_active_project. Key methods: create(), deleteProject(), getByUser(), getActive(), setActive(), getById().

task.js — TaskManager

Full CRUD for tasks persisted in vertex_tasks, including moveTask() to change a card's column status. Provides isOverdue() and isDueSoon() helpers and cascades deletes when a parent project is removed. Key methods: create(), update(), moveTask(), deleteTask(), getByProject(), getById().

notification.js — NotificationManager

Manages the in-app activity feed (vertex_notifications, capped at 50 entries), toast pop-ups, and the background reminder checker. The checker polls every 30 seconds via setInterval; it fires overdue alerts once per calendar day (keyed in sessionStorage) and custom reminders within a ±1-minute window.

board.js — BoardRenderer

Reads tasks for the active project, applies three independent filters (full-text search, priority, assignee), and renders the three Kanban columns. Implements the HTML5 Drag and Drop API (dragstart / dragover / drop) to let users move cards between columns, updating status in localStorage on drop.

modal.js — ModalController

Opens and closes the task-create/edit modal, project-create modal, task-detail view, and the global confirmation dialog. Validates required fields (title, due date, at least one assignee) and populates the assignee-autocomplete list.

app.js — App (Controller)

Central controller that wires all DOM events: login/register forms, search input, priority and assignee filter dropdowns, notification bell, sidebar project clicks, and the responsive sidebar toggle. Also seeds demo tasks automatically for every newly registered account.

6.2 Key algorithms or code snippets

1. Password Hashing — simpleHash() in auth.js

Passwords are never stored as plain text. A deterministic bitwise hash (djb2 variant) converts each password string into a base-36 integer before writing to localStorage. While not production-grade cryptography, this is sufficient for a client-only demo application and prevents casual plain-text exposure in browser storage.

```
function simpleHash(str) {   let hash = 0;   for (let i = 0; i < str.length; i++) {
const char = str.charCodeAt(i);     hash = ((hash << 5) - hash) + char;     hash |= 0;
// clamp to 32-bit integer   }   return hash.toString(36);   // base-36 string }
```

2. HTML5 Drag and Drop — BoardRenderer in board.js

Task cards carry the draggable="true" attribute. The dragstart event stores the task UUID in the DataTransfer object. Each Kanban column listens for dragover (to permit the drop) and drop (to read the UUID and call TaskManager.moveTask()). A drag-over CSS class provides real-time visual feedback during the drag.

```
card.addEventListener('dragstart', e => {   card.classList.add('dragging');
e.dataTransfer.setData('text/plain', card.dataset.taskId);
e.dataTransfer.effectAllowed = 'move'; });  col.addEventListener('drop', e => {
e.preventDefault();   const taskId = e.dataTransfer.getData('text/plain');   const
newStatus = col.dataset.status; // "todo" | "inprogress" | "done"   if (taskId &&
newStatus) {     TaskManager.moveTask(taskId, newStatus);     BoardRenderer.render();
} });
```

3. Search and Filter Pipeline — BoardRenderer.render() in board.js

Each board render retrieves all tasks for the active project and applies up to three independent filters in sequence: full-text search across title and description, exact-match priority filter, and exact-match assignee filter. The resulting array is mapped to HTML card strings and injected into each column.

```
let tasks = TaskManager.getByProject(projectId);  // 1. Full-text search (title +
description) if (searchQuery) {   const q = searchQuery.toLowerCase();   tasks =
tasks.filter(t =>     t.title.toLowerCase().includes(q) ||
t.description.toLowerCase().includes(q)); } // 2. Priority exact-match if
(filterPriority)   tasks = tasks.filter(t => t.priority === filterPriority);  // 3.
Assignee exact-match if (filterAssignee)   tasks = tasks.filter(t =>
Array.isArray(t.assignees)       ? t.assignees.includes(filterAssignee)       :
t.assignee === filterAssignee);
```

4. Reminder Checker — setInterval in notification.js

On login, NotificationManager.startReminderChecker() launches a 30-second polling loop. Each tick scans every task: if overdue and not alerted today (sessionStorage key), it fires an overdue toast. If a task has a reminder timestamp within ±1 minute of now and has not fired this session, it fires a reminder toast and logs a feed notification.

```
function checkReminders() {   const now = new Date();
TaskManager.getAll().forEach(task => {      // --- Overdue alert (once per calendar day)
---     if (TaskManager.isOverdue(task)) {       const key =
`vertex_overdue_${task.id}_${now.toDateString()}`;       if
(!sessionStorage.getItem(key)) {         sessionStorage.setItem(key, '1');
showToast('reminder', `"${task.title}" is overdue!`, task.id);       }     }      // ---
Custom reminder (+/- 1 min window) ---     if (task.reminder) {       const diff = (new
Date(task.reminder) - now) / 60000;       const remKey = `vertex_rem_${task.id}`;
if (diff >= -1 && diff <= 1 && !sessionStorage.getItem(remKey)) {
sessionStorage.setItem(remKey, '1');         showToast('reminder', `Reminder:
"${task.title}"`, task.id);       }     }   }); }
```

5. XSS Prevention — escHtml() in board.js and app.js

All user-supplied strings (task titles, descriptions, project names, assignee names) pass through escHtml() before being written into the DOM as innerHTML. The function assigns the raw string as element.textContent (the browser escapes it automatically) and reads back the sanitised innerHTML, blocking stored XSS with no external library.

```
function escHtml(str) {   const div = document.createElement('div');   div.textContent =
str;   // browser escapes &, <, >, ", '   return div.innerHTML;    // safe HTML string }
```

6.3 User interfaces

The following screenshots illustrate the final user interface of the Vertex KANBAN System.

The Login screen presents the Vertex KANBAN logo, a tab bar to switch between Login and Register, an Email field, a Password field, and a "Sign In" button. Existing users enter their credentials here to access their dashboard.

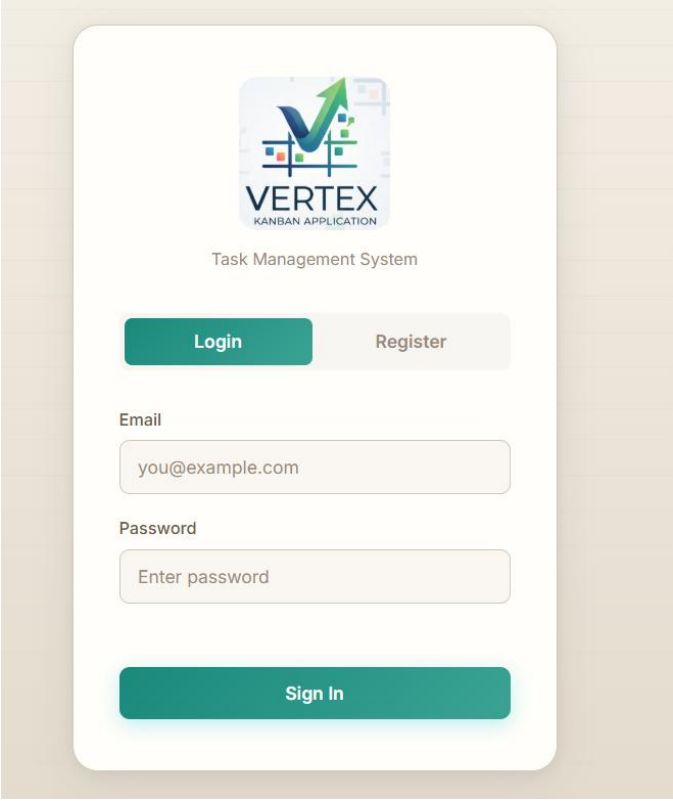


Figure 6.1 — Login Page

The Register screen collects a Full Name, Email, and Password (minimum 4 characters). Submitting the form creates a new account stored in browser localStorage.

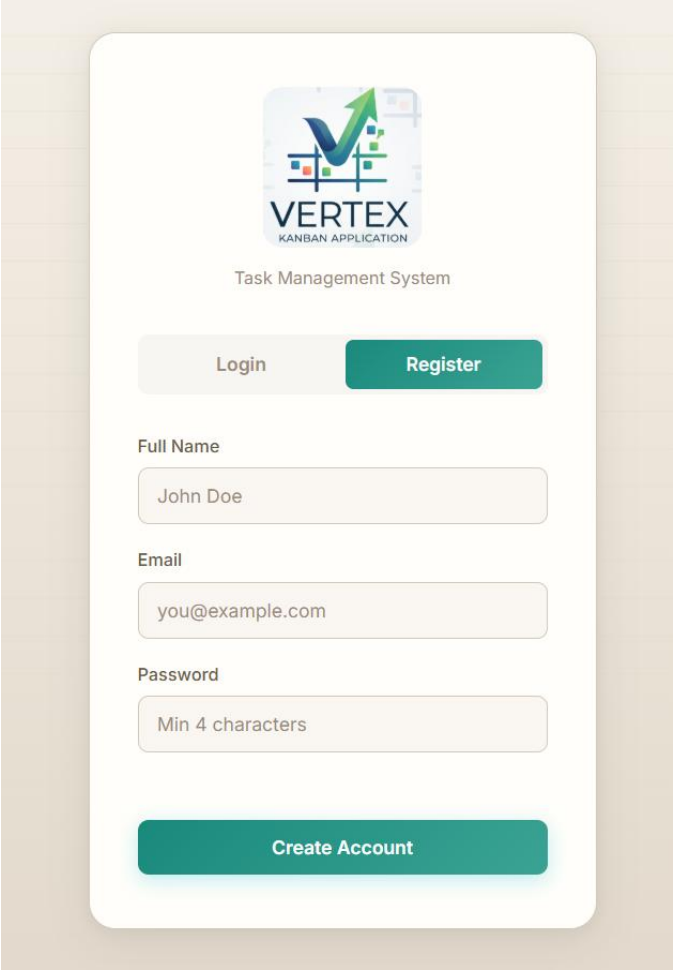


Figure 6.2 — Registration Page

The main board displays three columns: TO DO (5 tasks), IN PROGRESS (3 tasks), and DONE (4 tasks). Each card shows the task title, description, priority badge, due date, and assigned-member avatar. The top bar provides a search field and dropdowns to filter by priority or member. A notification bell shows the unread count.

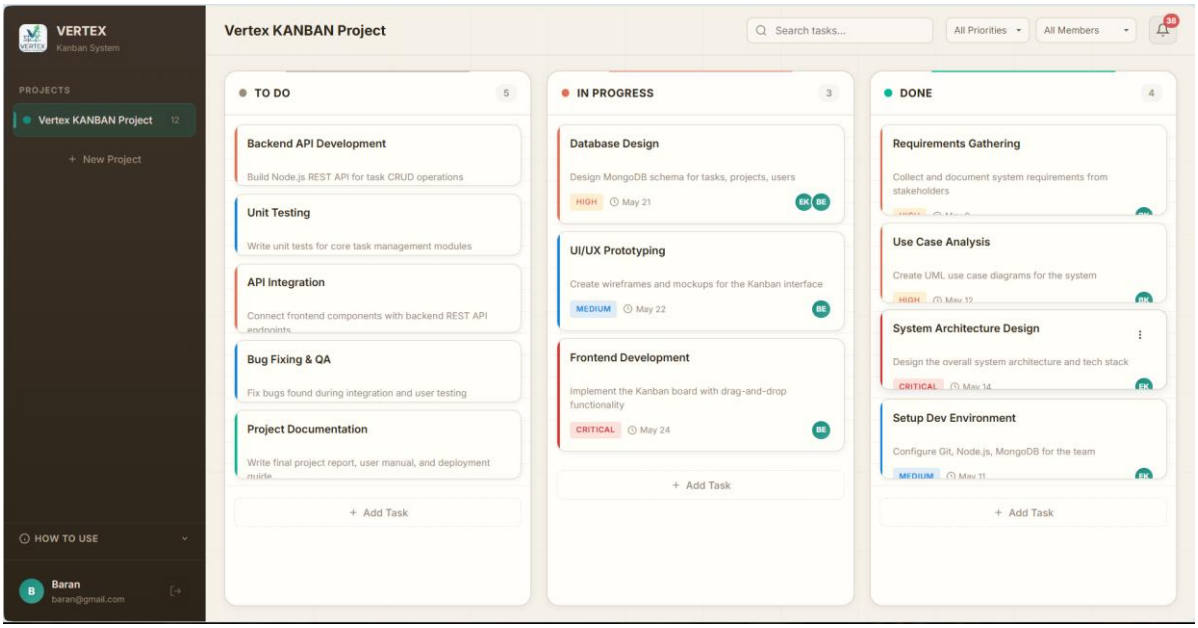


Figure 6.3 — Kanban Board View

The "New Project" modal allows users to enter a Project Name, an optional Description, and select a colour from six presets before clicking "Create Project".

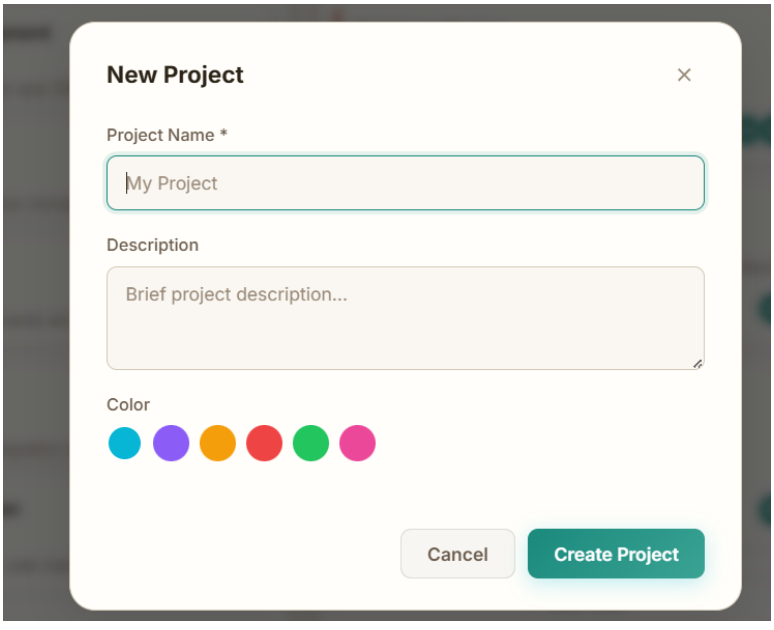


Figure 6.4 — New Project Modal

6.4 Open-source code

No third-party libraries or open-source packages were used in this project. The application is built entirely from scratch using native browser APIs. The Inter typeface is loaded from Google Fonts under the Open Font License (OFL). SVG icons used in the UI are custom-drawn inline SVG elements.

Chapter 7: Testing

7.1 Unit testing approach
7.2 Test Cases

TC ID	Test Case	Expected Result	Status
TC-001	Register with valid data	User stored in vertex_users	PASS
TC-002	Register with duplicate email	Error message displayed	PASS
TC-003	Login with correct credentials	Session created in localStorage	PASS
TC-004	Login with wrong password	'Incorrect password' message shown	PASS
TC-005	Create project with no name	Project not created, error shown	PASS
TC-006	Create task with empty title	Validation error, save prevented	PASS
TC-007	Create task with all fields	Card appears in the correct column	PASS
TC-008	Drag task from To Do to Done	Status updated in localStorage	PASS
TC-009	Search 'API'	Only matching cards shown	PASS
TC-010	Filter by Priority = Critical	Only critical cards shown	PASS
TC-011	Delete project	All associated tasks removed	PASS

TC-012	XSS: title='<script>alert(1)</script>'	Rendered as text, not executed	PASS
TC-013	Refresh page	All projects and tasks persist	PASS
TC-014	Past due date task	Overdue indicator shown in red	PASS
TC-015	Notification bell interaction	Unread badge shown; click clears badge	PASS

7.3 Acceptance Testing

User Acceptance Testing (UAT) was conducted to ensure the Vertex KANBAN System effectively meets the daily workflow needs of small software teams. The testing heavily prioritized the core Kanban board functionalities and a friction-free user experience. Key acceptance scenarios successfully validated include:

- **Kanban Board Workflows:** Testers verified that tasks could be seamlessly dragged and dropped between the "To Do", "In Progress", and "Done" columns. Visual updates were immediate, and the underlying task status updated correctly without errors.

- **Data Persistence Without a Server:** Because the system operates as a zero-installation, backend-free tool, it was crucial to test state persistence. Tests confirmed that hard-refreshing the browser or closing the tab preserved all projects, tasks, and column positions exactly as they were left, thanks to the localStorage implementation.

- **Dynamic Board Filtering:** Users validated that the Kanban board could dynamically filter visible cards without breaking the layout. Filtering by specific task "Assignees" or "Priority" successfully hid irrelevant tasks while maintaining the proper column structure.

- **Workspace Management:** Acceptance criteria confirmed that users could quickly create new projects, customize them with color tags, and switch between multiple isolated Kanban boards without any cross-project data leakage.

Chapter 8: Conclusion

8.1 Project Summary

Vertex KANBAN System is a fully functional, browser-based task management application developed as the CMPE314 Software Engineering project by a team of four students. The system addresses the need for a lightweight, zero-installation Kanban tool for small software teams. Key deliverables include: a complete requirements specification (12 FRs, 8 NFRs), use case and sequence diagrams, activity diagrams, a class model, context and Level-1 DFDs, a working implementation in HTML/CSS/JS, and a test suite of 15 manual test cases with full pass results.

8.2 Challenges Faced & Lessons Learned

Throughout the development lifecycle, the team encountered several challenges, primarily stemming from architectural mismatches between our early system designs and the final codebase. By analyzing our early design notes, we identified the following struggles and takeaways:

- **Technology Stack Pivot & Architectural Mismatch:** Our initial system models (including Sequence Diagrams, Context Diagrams, and DFDs) were drafted under the assumption that we would build a traditional backend architecture with distinct microservices (Authentication Service, Task Service, Board Service, Notification Service) and separate database stores. Due to scope and timeline constraints, we pivoted to a pure Single-Page Application (SPA) using Vanilla JavaScript and localStorage.

- **Consolidating System Logic:** Because of the SPA pivot, the external services depicted in early design diagrams had to be internalized. All separate backend logic and data routing had to be consolidated into internal JavaScript IIFE modules (such as AuthManager and TaskManager). For example, the Notification Service, originally modelled as an external participant, had to be rebuilt as an internal browser feature.

- **Evolving Agile Requirements:** Adopting an iterative Agile methodology meant our requirements evolved during coding. Our Level-2 DFD originally outlined a dedicated "Sort and Prepare Results" process, which ended up being unnecessary in the final implementation. Additionally, useful features like filtering by "Assignee" were added late in development, causing a slight drift from the original specification.

- **The Need for Living Documentation:** The discrepancies between our planning models and our final code taught us a valuable software engineering lesson. We learned that design diagrams should not be static; they must be treated as "living documents" and updated iteratively alongside development to prevent them from falling out of sync with the actual implementation.

References

[1] MDN Web Docs. (2024). Using the HTML Drag and Drop API. https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API
[2] MDN Web Docs. (2024). Web Storage API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
[3] OWASP Foundation. (2021). Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
[4] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
[5] Google Fonts. Inter typeface. https://fonts.google.com/specimen/Inter

Appendices

Appendix A — Gantt Chart: See Lab 1 Gantt Chart (Project 1.jpg / Lab 1.jpg) in the repository.
Appendix B — UML Diagrams: See UML_Diagram.jpg, Lab4.jpg, Lab5.jpg in the repository.
Appendix C — DFDs: See Lab 6 .pdf (Level 0-1-2 DFDs) in the repository.
Appendix D — GitHub Repository: https://github.com/22314825/CMPE314_Project
Appendix E — Work Breakdown Table (see below):

Task ID	Task Name	Assigned
1	Requirements Gathering	Baran
2	Use Case Analysis	Baran
3	System Architecture Design	Efe
4	Database Design	Berk, Efe
5	UI/UX Prototyping	Berk
6	Setup Development Environment	Emirhan
7	Frontend Development	Berk
8	Backend Development	Efe
9	API Integration	Efe, Berk
10	Testing (Unit + Integration)	Berk, Emirhan
11	User Acceptance Testing	Emirhan, Baran
12	Bug Fixing	Efe, Berk
13	Deployment Preparation	Emirhan, Baran
14	Final Deployment	Baran
15	Project Documentation	Baran, Emirhan